

AI STACK DOCTOR v4

Technical Reference · Architecture & Setup

For Engineers, By Engineers.

Python 3.11+

Anthropic Claude

Flask + Gunicorn

SQLite3

DuckDuckGo

ReportLab

Render.com / Docker

MIT License · Open Source · github.com/dwnjuguna/AI-Stack-Doctor

ai-stack-doctor.onrender.com

Navigate by experience level

Jump to the section that matches your background — or read the full deck for complete architectural context.



Junior Dev

Slides 3–6

Setup, installation, first run.
Get it working in 15 minutes.

PREREQS:

*Python 3.11+, Anthropic API key,
Git basics*



Mid-Level Eng

Slides 7–10

Agent architecture, scoring
algorithm, API endpoints.
Integrate it into your stack.

PREREQS:

*REST API concepts, JSON/SQLite,
Python familiarity*



Senior Eng

Slides 11–14

Extend the tool — new
profiles, frameworks, local
LLMs, Docker deployment.

PREREQS:

*LLM API experience, Docker basics,
system design*



Principal / Architect

Full deck

End-to-end architecture
decisions, security posture,
enterprise deployment.

PREREQS:

*All of the above + compliance
awareness*

Full system architecture — every component and connection

BROWSER / CLIENT

- ▶ dashboard.html — React-style vanilla JS, 44 companies, cohort filter
- ▶ intake_form.html — 4-persona intake, JSON download
- ▶ guide.html — Non-technical landing page
- ▶ security.html · legal.html

FLASK WEB SERVER

- ▶ dashboard_server.py — 31 routes · Auth · Scheduler · Waitlist
- ▶ scheduler.py — AgenticScheduler · AlertEngine · DigestEngine · ScheduleStore
- ▶ SQLite DB: ai_stack_history.db (audits · alerts · schedules)
- ▶ JSON: schedules.json · api_keys.json · audit_log.jsonl

AGENT ENGINE

- ▶ ai_stack_health_agent_v3.py — 6 tools · SYSTEM_PROMPT · ROI_FRAMEWORK
- ▶ pdf_export.py — ReportLab dark-themed PDF generator
- ▶ intake_reader.py — JSON parser · Persona router · Audit runner
- ▶ COMPANY_INTEL: 44 profiles · 5 regions · 28 industries

EXTERNAL SERVICES

- ▶ Anthropic API — claude-sonnet-4 · max_tokens:16000 · Tool use mode
- ▶ DuckDuckGo (default) — ddgs · Privacy-first · Configurable
- ▶ Google / Bing — Optional · env-configured
- ▶ SMTP / Webhook — Slack · Teams · Email alerts

The agentic loop — how 6 tools work in sequence

Claude operates in tool-use mode — calling each tool, receiving results, and synthesizing a 100-point score with a full written report.

01

`detect_ai_stack()`

8 targeted searches × 7 domains — job postings, blogs, vendor sites, GitHub

~8 searches

02

`research_stack_health()`

Deprecations, security advisories, vendor stability, G2/Gartner sentiment

~5 searches

03

`check_ai_integrations()`

Pipeline health, data flows, model serving, observability gaps

~4 searches

04

`audit_governance_and_ownership()`

14 compliance frameworks + ROI context + REWIRED industry weighting

~7 searches

05

`detect_redundancies_and_gaps()`

Capability overlaps, duplicate tools, wasted spend estimation

~4 searches

06

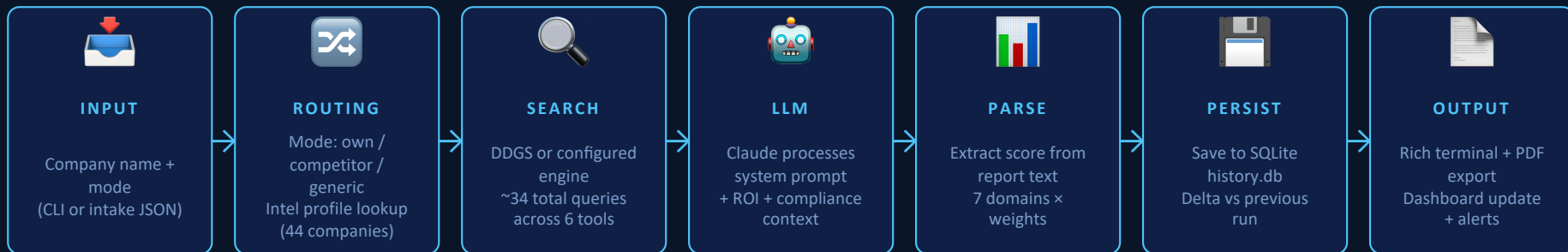
`benchmark_against_peers()`

3-company comparison from 44 pre-loaded intelligence profiles

~6 searches

```
# run_agent() main loop
messages = [{
    "role": "user",
    "content": company
}]
while True:
    response = claude.create(
        model="claude-sonnet-4",
        tools=tools,
        messages=messages
    )
    if stop == "tool_use":
        results = []
        for block in response:
            result = run_tool(
                block.name,
                block.input
            )
            results.append(result)
        messages.append(results)
    elif stop == "end_turn":
        return parse(response)
    # → 100-pt score + report
```

Data flow — from user input to scored report



```
run_agent("Stripe","competitor") → COMPANY_INTEL.get(company) → _search_ddg(query) →  
client.messages.create(tools=tools) → parse_overall_from_report(text) → save_to_history(company, score)
```

DATA BOUNDARY: Only the company name leaves your machine — as a search query. All scoring, history, and reports are computed and stored locally using your own ANTHROPIC_API_KEY. No telemetry. No external logging.

Complete tech stack — every dependency and why it was chosen

Five of twelve packages are Python stdlib. Where third-party packages are used, they are the canonical choice for their role.

Package	Version	Role	Why This Choice
<code>anthropic</code>	<code>>=0.40.0</code>	Core AI Engine	Official Anthropic Python SDK. Tool use, streaming, message API.
<code>ddgs</code>	<code>>=7.0</code>	Default Search	DuckDuckGo — privacy-first, no API key needed. Renamed from <code>duckduckgo_search</code> .
<code>flask</code>	<code>>=3.0</code>	Web Server	Lightweight WSGI framework for the 31-route API server and static file serving.
<code>gunicorn</code>	<code>>=22.0</code>	WSGI Server	Production WSGI server. Multi-worker. Required for Render.com deployment.
<code>rich</code>	<code>>=13.0</code>	CLI Display	Beautiful terminal output — progress spinners, colored panels, tables.
<code>reportlab</code>	<code>>=4.0</code>	PDF Export	Dark-themed audit report generation. No external dependencies.
<code>sqlite3</code>	<code>stdlib</code>	Database	Zero-config database for audit history, alerts, scheduler state.
<code>threading</code>	<code>stdlib</code>	Scheduler	Background daemon threads for agentic scheduler. Zero external deps.
<code>smtplib</code>	<code>stdlib</code>	Alerts	Email delivery for scheduler alerts. SMTP + STARTTLS.
<code>urllib.request</code>	<code>stdlib</code>	Webhooks	Webhook delivery for Slack/Teams/custom endpoints.
<code>hashlib/secrets</code>	<code>stdlib</code>	Auth	SHA-256 hashing for API keys. <code>secrets.token_urlsafe</code> for generation.

Prerequisites — what you need before you start

Verify each dependency before cloning. Full setup takes under 15 minutes for a developer already familiar with Python environments.

DEPENDENCY CHECKLIST

Python 3.11+	<code>python3 --version</code>	✓ Python 3.11.x
pip / pip3	<code>pip3 --version</code>	✓ pip 23+
Git	<code>git --version</code>	✓ Any version
Anthropic API Key	<code>console.anthropic.com</code>	✓ sk-ant-...
Internet access	<code>curl https://api.anthropic.com</code>	✓ HTTP 401

```
# Mac / Linux — set for current session
export ANTHROPIC_API_KEY="sk-ant-api03-..."
```

```
# Persist across sessions (~/.zshrc)
echo 'export ANTHROPIC_API_KEY=\
  "sk-ant-api03-..."' >> ~/.zshrc
source ~/.zshrc
```

```
# Windows (PowerShell)
$env:ANTHROPIC_API_KEY="sk-ant-..."
```

```
# Verify connectivity:
curl https://api.anthropic.com
# HTTP 401 = connected ✓ (key auth expected)
```

PATH A — Git Clone (Recommended)

```
git clone https://github.com/
dwnjuguna/AI-Stack-Doctor
cd AI-Stack-Doctor
pip3 install -r requirements.txt
python3 -c "import anthropic"
# No output = success ✓
```

PATH B — Dashboard Mode

```
python3 dashboard_server.py
# 🤖 Agentic scheduler started
# Running on http://127.0.0.1:5050
open http://localhost:5050
# CLI: python3 ai_stack_health_agent_v3.py
# API: python3 ai_stack_health_agent_v3.py
--api
```

PATH C — Render.com (Production)

```
# render.yaml pre-configured:
# build: pip install -r requirements.txt
# start: gunicorn dashboard_server:app
# Set ANTHROPIC_API_KEY env var
# Connect GitHub → Deploy
# Live in ~3 min : Auto-deploys on push
```

Configuration — environment variables and config files

SMTP is optional — omitting it disables email alerts but does not affect audit functionality. Search engine is swappable at runtime.

Variable	Required	Description
ANTHROPIC_API_KEY	Required	Your Anthropic API key — console.anthropic.com
SMTP_HOST	Optional	SMTP server for scheduler email alerts (e.g. smtp.gmail.com)
SMTP_PORT	Optional	SMTP port — default: 587
SMTP_USER	Optional	SMTP login username or email address
SMTP_PASSWORD	Optional	SMTP password or app-specific password
SMTP_FROM	Optional	From address for outgoing alert emails

```
# search_config.json (auto-created)
{
  "engine": "ddg",    // ddg | google | bing | serpapi | custom
  "key": "",         // Google API key (if google)
  "cx": ""          // Google CSE ID (if google)
}
# Switch: Settings modal · POST /api/config/search · edit file
```

```
# .env file alternative
# pip install python-dotenv
from dotenv import load_dotenv
load_dotenv()

# .env file contents:
ANTHROPIC_API_KEY=sk-ant-...
SMTP_HOST=smtp.gmail.com
SMTP_USER=you@gmail.com
SMTP_PASSWORD=abcd-efgh-ijkl
```


First run — step-by-step CLI walkthrough

The following walkthrough audits Stripe as a competitor. ~34 searches executed across 6 tools. Report ready in ~90 seconds.

```
$ python3 ai_stack_health_agent_v3.py
```

```
AI STACK DOCTOR v4.0 🤖
```

Select mode:

1. Audit YOUR company
2. Audit a COMPETITOR
3. Generic best-practices

> 2

Company name: > Stripe

Step 1 → Detecting AI Stack

[8 searches: job postings, blogs, GitHub]

Step 2 → Researching Stack Health

[5 searches: G2, deprecations, advisories]

Step 3 → Checking Integrations

[4 searches: pipelines, APIs, data flows]

Step 4 → Auditing Governance

[7 searches: compliance, regulatory]

Step 5 → Detecting Redundancies

[4 searches: overlaps, spend signals]

Step 6 → Benchmarking Peers

[Adyen, Nubank, Square from 44 profiles]

✓ Report complete: 90/100

Export? t=txt p=pdf b=both n=skip

> b

✓ TXT: ai_stack_stripe_20260626.txt

✓ PDF: ai_stack_report_stripe_20260626.pdf

Mode 2 — Competitor

External-signals-only audit using public data + 44 company intel profiles.

~34 Total Searches

Tools 01–06 run in sequence. rich renders live spinners and progress panels.

Score: 90/100

Claude synthesizes all tool results across 7 domains × weighted by industry context.

Score Persisted

Saved to SQLite with delta vs. previous run. Dashboard auto-updates on next load.

Dual Export (b)

TXT: plain text report. PDF: dark-themed branded report via ReportLab. Timestamped.

What's next : Agent Deep Dive & Extensions

Part 2 covers the agent internals, scoring algorithm, extensions, and MCP server integration.

01

System Prompt Architecture

8 injection blocks · ~3,400 tokens · ROI framework · Compliance

03

Scoring Algorithm

100-point rubric · what earns/loses points per domain

05

Adding Company Profiles

COMPANY_INTEL dict · full field reference · ground truth signals

07

Search Engine Config

DDG · Google · Bing · SerpAPI · custom engine interface

09

Industry Value Map (REWired)

9 sector profiles · McKinsey framework · Scaling Purgatory flag

02

Tool Definitions

JSON schemas · `run_tool()` dispatcher · all 6 tool signatures

04

History Tracking & Delta

SQLite schema · `save_to_history()` · trend API endpoints

06

Compliance Frameworks

GLOBAL_COMPLIANCE · extend the 14 built-in frameworks

08

Local LLM / Air-Gap Mode

Ollama · LM Studio · zero external calls

10

MCP Server

stdio free tier · Claude Desktop setup · 3 tools live now

System prompt architecture — what the agent knows before it starts

Total system prompt: ~3,400 tokens · Model input window: 200K tokens · Max output: 16,000 tokens · Each audit uses ~35K–80K total tokens

IDENTITY + MISSION

~200 tokens

Establishes the agent as an AI stack auditor. Sets tone, scope, and output format expectations.

ROI_FRAMEWORK

~400 tokens

9 domain ROI benchmarks. Gap costs, fix costs, payback periods. `get_roi_context()` called at runtime.

7 DOMAIN DEFINITIONS

~400 tokens

GenAI/LLMs, Agentic AI, ML, Data Eng, AI Platforms, MLOps, Cloud AI — each with scoring criteria and weight.

PRESCRIPTION_PRIORITY

~200 tokens

CRITICAL/HIGH/MEDIUM/LOW criteria. Quick Win examples per domain. `get_prescription_context()`

SCORING FRAMEWORK

~300 tokens

100-point system. Weights: $14+14+14+14+14+14+16=100$. Score bands: 80+ Healthy, 60–79 Needs Attention, <60 At Risk.

COMPANY INTEL BLOCK

~800 tokens

Pre-loaded signals for target company if in `COMPANY_INTEL` dict. Known tools, scoring signals, search routes.

14 COMPLIANCE FRAMEWORKS

~600 tokens

EU AI Act, GDPR+AI, US EO on AI, CCPA, HIPAA+AI, China GenAI, plus 8 high-priority frameworks. All mapped to scoring.

OUTPUT FORMAT SPEC

~400 tokens

Exact structure for every report section. Prescription format, priority matrix, confidence summary.

Tool definitions — the JSON schema Claude uses to call each function

All 6 tools follow the same schema pattern. The `run_tool()` dispatcher maps tool names to Python functions.

```
tools = [
    {
        "name": "detect_ai_stack",
        "description": "Multi-angle research to detect all
            AI tools across 7 domains – job postings,
            engineering blogs, vendor announcements,
            and pre-loaded company intelligence.",
        "input_schema": {
            "type": "object",
            "properties": {
                "company_name": {"type": "string"}
            },
            "required": ["company_name"]
        },
    },
    # research_stack_health
    #   → input: company_name
    # check_ai_integrations
    #   → input: company_name
    # audit_governance_and_ownership
    #   → input: company_name + industry
    # detect_redundancies_and_gaps
    #   → input: company_name
    # benchmark_against_peers
    #   → input: company_name + peers[]
]
```

```
# run_tool() dispatcher
# Maps tool name → Python function

def run_tool(
    name: str,
    inputs: dict
) -> str:

    tool_map = {
        "detect_ai_stack":
            detect_ai_stack,

        "research_stack_health":
            research_stack_health,

        "check_ai_integrations":
            check_ai_integrations,

        "audit_governance_and_ownership":
            audit_governance,
            # injects ROI + compliance

        "detect_redundancies_and_gaps":
            detect_redundancies,

        "benchmark_against_peers":
            benchmark_against_peers,
    }
    return tool_map[name](**inputs)
```

Scoring algorithm — how 100 points are calculated

Claude synthesizes all tool results against the scoring rubric. Score is extracted via regex and persisted to SQLite with delta vs. previous run.

Domain	Weight	What Earns Points	What Loses Points
GenAI / LLMs	14 pts	GPT-4, Claude, Gemini · Fine-tuning · RAG pipelines · Prompt engineering	No LLM deployment · Only pre-2023 models · No RAG layer
Agentic AI	14 pts	LangChain, CrewAI, AutoGPT · Custom agents · Tool use · Orchestration layers	Manual-only workflows · No autonomous task execution
Machine Learning	14 pts	PyTorch/TF/JAX · Active model training · Feature engineering · A/B testing	No ML training · Only SaaS APIs · No model ownership
Data Engineering	14 pts	Airflow, dbt, Spark · Feature stores · Real-time pipelines · Data warehouse	Siloed data · No pipeline orchestration · Manual ETL
AI Platforms	14 pts	Internal ML platform · Vertex/SageMaker/AzureML · Hugging Face · Model serving	No platform layer · Scientists deploy ad-hoc · No model registry
MLOps / LLMOps	14 pts	LangSmith, Arize, MLflow · Monitoring · CI/CD for models · Drift detection	No monitoring · Manual deployments · No LLM observability
Cloud AI Services	16 pts	AWS Bedrock/SageMaker · GCP Vertex · Azure OpenAI · Broad service adoption	Only self-hosted · No managed AI services · Single cloud lock-in

```
# Score extraction regex
pattern = r"OVERALL.*?HEALTH.*?SCORE.*?[:\s]+(\d{1,3})"
match = re.search(pattern, report, re.IGNORECASE|re.DOTALL)
return int(match.group(1)) if match else None
```

 **Healthy 80–100**

 **Needs Attention 60–79**

 **At Risk < 60**

History tracking & delta — how trend data is stored and compared

Every audit is persisted to SQLite. Delta vs. previous run is calculated automatically. Dashboard APIs expose trend and comparison data.

```
-- SQLite schema: ai_stack_history.db

CREATE TABLE IF NOT EXISTS audits (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  company     TEXT NOT NULL,
  mode        TEXT,
  report      TEXT,      -- full report text
  overall     INTEGER,    -- 0-100 score
  scores      TEXT,      -- JSON {domain:{score,total}}
  created_at  TEXT       -- ISO timestamp
);

CREATE TABLE IF NOT EXISTS alerts (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  company     TEXT,      type TEXT,
  severity    TEXT,      message TEXT,
  delta       INTEGER, created_at TEXT
);
```

```
# save_to_history() - called after every audit
def save_to_history(
    company, mode, report, score, scores):
    conn = sqlite3.connect("ai_stack_history.db")
    conn.execute("""
        INSERT INTO audits
        (company,mode,report,overall,scores,created_at)
        VALUES (?, ?, ?, ?, ?, ?) """,
        (company, mode, report, score,
         json.dumps(scores),
         datetime.now().isoformat()))
    conn.commit(); conn.close()

# get_last_report() - for delta comparison
def get_last_report(company) -> dict | None:
    # Returns {overall, scores, created_at}
    # of the most recent run for this company
```

GET `/api/trend/<company>`

Returns last 10 audit scores with delta vs. previous run.
Used for trend charts on the dashboard.

```
{
  "date": "2026-06-26",
  "overall": 90,
  "delta": "+3",
  "scores": {
    "GenAI": {
      "score": 13,
      "total": 14
    }
  }
}
```

GET `/api/compare?a=Stripe&b=Nubank`

Returns radar chart data for side-by-side 7-domain comparison.

```
{
  "labels": ["GenAI", "MLOps", ...],
  "a": [13, 12, ...],
  "b": [9, 8, ...]
}
```

GET `/api/audit-log`

Returns last 100 audit events from audit_log.jsonl —
append-only, tamper-evident.

JSONL: one event per line, never truncated

Adding new company profiles — the most impactful extension

The agent treats `known_stack` as HIGH-CONFIDENCE ground truth — bypassing web search for confirmed tools. Add your client here before running the audit.

```
# COMPANY_INTEL dict — ai_stack_health_agent_v3.py (~line 200)
# Add your company or client here:

COMPANY_INTEL["acme_corp"] = {
    "known_stack": [
        "PyTorch",          # confirmed ML framework
        "Anthropic Claude", # GenAI provider
        "Airflow",          # data pipeline
        "MLflow",           # experiment tracking
        "Snowflake",        # data warehouse
    ],
    "signals": {
        "genai_score": 12, # out of 14 — high (using Claude)
        "mlops_score": 8,  # out of 14 — medium (MLflow, no monitoring)
        "data_score": 11,  # out of 14 — good (Snowflake + Airflow)
        "cloud_score": 13, # out of 16 — strong (multi-cloud)
    },
    "industry": "fintech",      # 28 options: see INDUSTRY_LIST
    "size":      "mid",         # startup | mid | large | enterprise
    "region":    "us",          # us | europe | asia | latam | africa
    "hq":        "San Francisco, CA",
    "search_routes": {
        "engineering_blog": "https://engineering.acmecorp.com",
        "github_org":       "acme-corp",
    },
    "peers": ["Stripe", "Plaid", "Marqeta"],
}

# Then run:
# python3 ai_stack_health_agent_v3.py → type "acme corp"
```

known_stack

`list[str]`

Confirmed tools — treated as ground truth. Bypasses web search for these.

signals.genai_score

`int (0–14)`

Pre-set domain score. Agent uses as starting point, may adjust based on new signals.

industry

`str`

One of 28 industry options. Activates REWIRED industry weighting in Tool 04.

size

`str`

startup | mid | large | enterprise. Filters compliance framework applicability.

region

`str`

us | europe | asia | latam | africa. Routes compliance and benchmark peers.

search_routes

`dict`

Optional custom search query routing — engineering blog URL, GitHub org name.

peers

`list[str]`

Benchmark targets for Tool 06. Must match keys in `COMPANY_INTEL`.

Adding compliance frameworks — extend the 14 built-in frameworks

`get_compliance_context(industry, region, size)` filters and injects active frameworks into every audit automatically.

```
# GLOBAL COMPLIANCE dict — ai_stack_health_agent_v3.py
# Currently 14 frameworks. Add yours here:

GLOBAL_COMPLIANCE["ISO 27001_AI"] = {
    "name": "ISO 27001 — AI Information Security Addendum",
    "priority": "HIGH", # CRITICAL | HIGH | MEDIUM
    "geographies": ["global"],
    "applies_to": ["enterprise", "mid"],
    "industries": ["all"],

    "key_requirements": [
        "AI system inventory and classification",
        "Risk assessment for AI-specific threats",
        "Incident response for AI model failures",
        "Supplier AI security requirements",
    ],

    "audit_questions": [
        "Do you have an inventory of all AI systems processing sensitive data?",
        "Is there an AI-specific incident response playbook?",
        "Are third-party AI vendors assessed for security compliance?",
    ],

    "max_fine": "Certification loss + contractual penalties",
    "effective_date": "2024-02-01",
    "reference": "https://www.iso.org/standard/27001",
}
```

CRITICAL

EU

EU AI Act (2024)

Risk-based classification · High-risk AI obligations

CRITICAL

EU

GDPR + AI

Personal data in AI training and inference

CRITICAL

US

US EO on AI

Federal AI safety and security standards

CRITICAL

US

CCPA / CPRA

California consumer AI data rights

CRITICAL

US

HIPAA + AI

Healthcare AI and PHI handling

CRITICAL

CN

China GenAI Regs

Generative AI service provider requirements

HIGH

GLOBAL

NIST AI RMF

AI risk management framework

HIGH

GLOBAL

ISO 42001

AI management system standard

Search engine configuration — DDG, Google, Bing, SerpAPI, or custom

Switch search engines at runtime — no restart required. Three configuration methods supported.

Default · Free

DuckDuckGo

engine: "ddg"

✓ Privacy-first · No API key needed · Zero config

✗ Rate limited at high volume

\$5 / 1,000 queries

Google Custom Search

engine: "google"

✓ High quality · Reliable · Huge index · Best results

✗ Requires GOOGLE_API_KEY + GOOGLE_CSE_ID

Paid · Azure

Bing Search API

engine: "bing"

✓ Good global coverage · Reliable · Structured JSON

✗ Requires BING_API_KEY via Azure portal

\$50/month minimum

SerpAPI

engine: "serpapi"

✓ Multiple engines · Structured output · Consistent

✗ Requires SERPAPI_KEY · Most expensive option

```
# Method 1 — Edit search_config.json directly
echo '{"engine":"google","key":"AIza...", "cx":"017..."}' \
> search_config.json
```

```
# Method 2 — API endpoint
curl -X POST localhost:5050/api/config/search \
-H "Content-Type: application/json" \
-d '{"engine":"google","key":"AIza...", "cx":"017..."}'
```

```
# Method 3 — Dashboard Settings modal
# Settings → Search Engine → Select + Save
```

```
# Custom search engine — implement the interface
# in ai_stack_health_agent_v3.py:
```

```
def _search_custom(
    query: str,
    max_results: int
) -> list[dict]:
    results = my_enterprise_search(query)
    return [
        {"title": r.title,
         "body": r.snippet,
         "href": r.url}
        for r in results[:max_results]
    ]
# Then add to SEARCH_ENGINES dict
```

Local LLM mode — run fully air-gapped with Ollama or LM Studio

Use case: privacy-maximum mode, cost control, or testing without API spend. Local mode means ZERO data leaves your machine.

USE CASE

Air-gap deployments · Privacy-maximum mode · Cost control · Testing without API spend

ZERO data leaves your machine — company name stays local, no Anthropic API calls.

```
# Ollama setup (Mac / Linux)
curl -fsSL https://ollama.com/install.sh | sh

# Pull a capable model:
ollama pull llama3.1:70b    # best tool use support
ollama pull mistral:7b      # faster, smaller
ollama pull qwen2.5:72b     # strong reasoning

# Start the server:
ollama serve
# → http://localhost:11434

# Verify:
curl http://localhost:11434/v1/models

# LM Studio: download at lmstudio.ai
# → Local Server tab → Start Server
# Default port: http://localhost:1234
```

```
# Patch ai_stack_health_agent_v3.py
# Replace the Anthropic client init:

import anthropic

client = anthropic.Anthropic(
    base_url="http://localhost:11434",
    api_key="ollama", # any string works
)

# Replace model name:
model="llama3.1:70b" # best tool use
# or model="mistral:7b"
```

llama3.1:70b



Best tool use support — recommended for ASD

qwen2.5:72b



Strong reasoning · Good alternative

mistral:7b



Fast · Lighter · May need --api flag

Any GGUF model



Via LM Studio · Tool use varies by model

Industry Intelligence — REWIRED framework integration

Every audit is weighted by industry context. `INDUSTRY_VALUE_MAP` encodes 9 sector profiles drawn from the McKinsey REWIRED framework (Lamarre, Smaje & Zemmell).

Fintech

Machine Learning · Data Engineering · MLOps

Healthcare

Governance · Machine Learning · Data Engineering

E-commerce

GenAI/LLMs · Data Engineering · Agentic AI

Media

GenAI/LLMs · Agentic AI · Cloud AI

Enterprise Software

Agentic AI · GenAI/LLMs · AI Platforms

Semiconductors

AI Platforms · Machine Learning · Cloud AI

Logistics

Machine Learning · Data Engineering · Agentic AI

Telecom

Data Engineering · MLOps · Machine Learning

Social Media

GenAI/LLMs · Machine Learning · Agentic AI

HOW IT WORKS

When `audit_governance_and_ownership()` (Tool 04) is called, `get_industry_context(industry)` injects sector-specific priority weighting into the system prompt. Fintech audits weight governance and data engineering higher. Media audits weight GenAI and agentic AI to the top. Prescriptions and the priority matrix adapt accordingly.

⚠️ **Scaling Purgatory Flag:** Detects companies stuck between Scaling and Optimizing maturity stages — a common trap where AI programs stall after initial deployment.

MCP Server — Connect Any AI Client

AI Stack Doctor exposes a Model Context Protocol (MCP) server — the open standard that lets AI assistants call external tools directly.

Tier	Transport	Works With	Status
Free	stdio	Claude Desktop only	✔ Live — ships with repo
Pro	Streamable HTTP	LangChain · LlamaIndex · OpenAI Assistants · CrewAI · custom agents	🔒 Coming soon

FREE TIER — 3 TOOLS OVER STDIO

`run_audit(company_name, mode)`

→ Full 90-second audit on any company

`get_audit_history(company_name, limit)`

→ Recent scores and trend data

`list_benchmarks(region, industry)`

→ All 44 company profiles with scores

```
// Claude Desktop config
// ~/Library/Application Support/
//   Claude/claude_desktop_config.json
{
  "mcpServers": {
    "ai-stack-doctor": {
      "command": "python",
      "args": ["-m", "mcp_server.server"],
      "cwd": "/path/to/AI-Stack-Doctor",
      "env": {
        "ANTHROPIC_API_KEY":
          "sk-ant-your-key-here"
      }
    }
  }
}

# Usage examples:
# "Audit Stripe's AI stack"
# "Show me fintech benchmarks"
# "What audits have I run recently?"

# Files: mcp_server/server.py
#         mcp_server/schemas.py
```